

Conexión de Datos Maestra

HTML → Node.js → MongoDB Atlas

Desarrollo e Implementación Completa de la Arquitectura del Sistema Hospitalario

Fase 1: Configuración en la Nube

Preparando la base de datos distribuida en MongoDB Atlas para aceptar conexiones externas.

| Liberar la IP en MongoDB Atlas

Por motivos de seguridad, las bases de datos en la nube rechazan conexiones desconocidas. Es obligatorio abrir el canal de acceso.



Network Access

Ubicado bajo el menú de seguridad.
Controla las políticas del Firewall
perimetral de tu cluster.



Acceso Global

Al añadir la IP asignada **0.0.0.0/0**,
permitimos que plataformas en la
nube cambiantes como Project IDX se
conecten sin restricciones.



Confirmación

Se guarda el cambio y se espera un
minuto a que el estado pase de
'Pending' a 'Active'.

Obtener la Cadena de Conexión (URI)

La URI es la dirección cifrada que le indica a nuestro código Backend dónde guardar los registros de los pacientes.

Pasos de Extracción:

- ➡ Ir a la pestaña principal de **Database** en Atlas.
- ➡ Dar clic sobre el botón azul de **Connect** del Cluster.
- ➡ Seleccionar el conector nativo por medio de la opción **Drivers**.
- ➡ Copiar la cadena generada que inicia con `mongodb+srv://...`

```
# Estructura típica de la cadena copiada:  
mongodb+srv://:@cluster.rvzjeba.mongodb.net/hospital #  
REGLA: Debes sustituir por tu clave real
```


Fase 2: Consola de comandos en IDX

Instalación de paquetes de node estructurando el entorno de trabajo.

Inicialización y Entorno Local

Comandos requeridos para construir la arquitectura base y arrancar el servidor visual del sitio web.

1. Estructurar el Proyecto

Genera en automático el manifiesto de configuración package.json en la raíz.

```
npm init -y
```

2. Servidor de Desarrollo

Descarga la herramienta encargada de renderizar los archivos HTML en tiempo real.

```
npm install lite-server --save-dev
```

Automatización de Arranque

Modificación del manifiesto de configuración para vincular comandos directos de ejecución de manera ágil.

1

Línea Única

Inyección directa vía Node

Asignación de "start"

Mediante esta instrucción de consola, se sobrescribe el bloque de ejecuciones internas para activar la web con el comando corto tradicional:

```
node -e "const fs = require('fs'); const pkg =
JSON.parse(fs.readFileSync('package.json')); pkg.scripts =
{ start: 'lite-server' }; fs.writeFileSync('package.json',
JSON.stringify(pkg, null, 2));"
```

Instalación de Librerías del Puente Backend

Se descargan las dependencias core encargadas de enlazar de forma nativa los lenguajes frontend con MongoDB Atlas.

- > **Express:** Framework minimalista enfocado a construir rutas de comunicación API (Endpoints).
- > **Mongoose:** Controlador encargado de modelar objetos JavaScript en esquemas válidos de Base de Datos.
- > **CORS:** Cabecera obligatoria de intercambio de recursos cruzados, habilitando peticiones multipartes de red.

```
npm install express mongoose cors dotenv
```


Fase 3: Desarrollo de Código y Arquitectura

Programación de la lógica del Servidor en Node.js y las peticiones interactivas cliente.

El Servidor Backend: server.js

Este archivo centraliza el motor de base de datos y define las rutas receptoras:

```
JS server.js
JS server.js > ...
1  const express = require('express');
2  const mongoose = require('mongoose');
3  const cors = require('cors');
4
5  const app = express();
6  app.use(express.json());
7  // NUEVA LÍNEA: Esto le dice a tu backend que si alguien entra a la raíz, cargue tu HTML
8  app.use(express.static(__dirname));
9  app.use(cors()); // Importante para que tu HTML pueda comunicarse con este backend
10
11 // URL de tu Base de Datos MongoDB (Reemplaza con tu cadena real)
12 const MONGO_URI = "mongodb+srv://kevin:Kevin12345@cluster0.rvzjeba.mongodb.net/hospital?appName=Cluster0";
13
14 mongoose.connect(MONGO_URI)
15   .then(() => console.log("¡Conectado exitosamente a MongoDB! 🚀"))
16   .catch(err => console.error("Error al conectar a MongoDB:", err));
17
18 // Esquema basado exactamente en tu formulario HTML
19 const PacienteSchema = new mongoose.Schema({
20   nombre: String,
21   edad: String,
22   sexo: String,
23   ocupacion: String,
24   diagnostico: String,
25   tratamiento: String
26 });
27 const Paciente = mongoose.model('Paciente', PacienteSchema);
28
29 // Ruta POST para recibir y guardar un nuevo paciente
30 app.post('/api/pacientes', async (req, res) => {
31   try {
32     const nuevoPaciente = new Paciente(req.body);
33     await nuevoPaciente.save();
34     res.json({ mensaje: "Paciente guardado en MongoDB correctamente" });
35   } catch (error) {
36     res.status(500).json({ error: "No se pudo registrar en la base de datos" });
37   }
38 });
39
40 const PORT = 5000;
41 app.listen(PORT, () => console.log(`Servidor Backend corriendo en http://localhost:${PORT}`));
```

Lógica del Cliente: Script de Envío Completo

Esta es la función agregarPaciente() completa que valida y transmite los objetos hacia MongoDB Atlas:

```
// AGREGAR PACIENTE Y GUARDAR EN MONGODB //  
async function agregarPaciente(){  
  
    let nombre = document.getElementById("nombre").value;  
    let edad = document.getElementById("edad").value;  
    let sexo = document.getElementById("sexo").value;  
    let ocupacion = document.getElementById("ocupacion").value;  
    let diagnostico = document.getElementById("diagnostico").value;  
    let tratamiento = document.getElementById("tratamiento").value;  
  
    if(  
        nombre == "" || edad == "" || sexo == "" ||  
        ocupacion == "" || diagnostico == "" || tratamiento == ""  
    ){  
        alert("⚠️ Completa todos los campos");  
        return;  
    }  
  
    const datosPaciente = {  
        nombre,  
        edad,  
        sexo,  
        ocupacion,  
        diagnostico,  
        tratamiento  
    };  
  
    try {  
  
        const respuesta = await fetch('/api/pacientes', {  
            method: 'POST',  
            headers: {  
                'Content-Type': 'application/json'  
            },  
            body: JSON.stringify(datosPaciente)  
        });  
    }  
}
```


Lógica del Cliente: Buscador Dinámico

Módulo programado en JavaScript que rastrea y filtra los expedientes en pantalla al instante.

```
0  try {
1
2      const respuesta = await fetch('/api/pacientes', {
3          method: 'POST',
4          headers: {
5              'Content-Type': 'application/json'
6          },
7          body: JSON.stringify(datosPaciente)
8      });
9
10     const resultado = await respuesta.json();
11
12     if (respuesta.ok) {
13         let nuevaCard = document.createElement("div");
14         nuevaCard.classList.add("card");
15         nuevaCard.innerHTML = `
16             <h2>${nombre}</h2>
17             <p><strong>Edad:</strong> ${edad}</p>
18             <p><strong>Sexo:</strong> ${sexo}</p>
19             <p><strong>Ocupación:</strong> ${ocupacion}</p>
20             <p><strong>Diagnóstico:</strong> ${diagnostico}</p>
21             <p><strong>Tratamiento:</strong> ${tratamiento}</p>
22         `;
23
24         document.getElementById("listaPacientes").appendChild(nuevaCard);
25
26         /* LIMPIAR FORMULARIO */
27         document.getElementById("nombre").value = "";
28         document.getElementById("edad").value = "";
29         document.getElementById("sexo").value = "";
30         document.getElementById("ocupacion").value = "";
31         document.getElementById("diagnostico").value = "";
32         document.getElementById("tratamiento").value = "";
33
34         alert("✅ " + resultado.mensaje);
35     } else {
36         alert("❌ Error al guardar en el servidor: " + resultado.error);
37     }
38 } catch (error) {
39     console.error("Error en la petición:", error);
40     alert("⚠️ No se pudo conectar con el servidor backend. ¿Está encendido?");
41 }
42 }
```

Mecánica del Algoritmo:

- ⚡ **Evento keyup:** Se dispara de inmediato al teclear en el campo del filtro.
- 🔧 **Normalización:** Convierte cadenas de texto a minúsculas, neutralizando errores de tipeo.
- 🔽 **Inclusión por CSS:** Conmuta la propiedad display para ocultar las tarjetas que no hagan match.

Análisis de los Procesos Core del Servidor

Mapa lógico de responsabilidades estructurales configuradas en nuestro ecosistema web:

Línea de Código

Misión Estratégica en el Proyecto

`express.static(__dirname)`

Transforma la carpeta de IDX en una ubicación pública. Levanta tus HTML sin usar servicios externos.

`app.use(cors())`

Desactiva las restricciones de seguridad que impiden cruzar datos locales entre el puerto web y la base de datos.

`mongoose.model('Paciente', ...)`

Define el contrato de almacenamiento. Asegura que los campos coincidan perfectamente con tu formulario.

`app.post('/api/pacientes')`

Crea el túnel de entrada. Escucha el JSON enviado por fetch e invoca la operación de grabado en la nube.

Puesta en Marcha Final

Para que tu ecosistema funcione, debes activar ambas capas del proyecto abriendo dos pestañas en Project IDX:

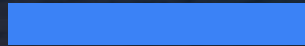
```
Terminal 1: npm start # Levanta el frontend visual  
(puerto 3000)
```

```
Terminal 2: node server.js # Levanta el cerebro y  
base de datos (puerto 5000)
```

El formulario web empaqueta la información, la manda mediante la función asíncrona por el puerto 5000 y el backend la guarda directo en MongoDB Atlas.

¿Preguntas?

Sistema Hospitalario Digital - Proyecto Final de
Programación



Alumno: Kevin Raul Gonzalez Chavez

Número de Control: 23308051280594

Especialidad: Programación • Grupo: 4-I